# AED2 - Atividade Teórica - Aula 3

Aluno: João Marcelo Franca Batista

## 1) Shellsort, Quicksort e Heapsort em {22, 35, 2, 1, 13, 7}

### Shellsort (h = 3, 2 e 1)

Para cada valor de h, o vetor é separado em sublistas formadas pelos elementos que estão a h posições de distância um do outro (posições contadas a partir de 0). Cada sublista é ordenada por inserção e, depois, o h diminui. No último passo (h = 1) é um Insertion Sort comum.

| h | Sublistas (posições) e como ficam depois de ordenadas | Vetor após o passo |
|---|---|---|
| 3 | posições 0, 3: `22 1` → `1 22`<br>posições 1, 4: `35 13` → `13 35`<br>posições 2, 5: `2 7` (já ordenada) | `1 13 2 22 35 7` |
| 2 | posições 0, 2, 4: `1 2 35` (já ordenada)<br>posições 1, 3, 5: `13 22 7` → `7 13 22` | `1 7 2 13 35 22` |
| 1 | posições 0, 1, 2, 3, 4, 5: `1 7 2 13 35 22` → `1 2 7 13 22 35` | `1 2 7 13 22 35` |

Detalhe do passo h = 1 (inserção de cada elemento, a partir do vetor 1 7 2 13 35 22):

| Posição i | Elemento | O que acontece | Vetor |
|---|---|---|---|
| 1 | 7 | já está no lugar | `1 7 2 13 35 22` |
| 2 | 2 | desloca para a posição 1 | `1 2 7 13 35 22` |
| 3 | 13 | já está no lugar | `1 2 7 13 35 22` |
| 4 | 35 | já está no lugar | `1 2 7 13 35 22` |
| 5 | 22 | desloca para a posição 4 | `1 2 7 13 22 35` |

`Resultado final do Shellsort: 1 2 7 13 22 35`

### Quicksort (pivô: mediana de três)

Pivô escolhido em cada intervalo: a **mediana de três** valores — o primeiro, o do meio e o último do intervalo (o mesmo critério usado no código da atividade prática). Partição de Hoare: um índice esquerdo avança enquanto o elemento for menor que o pivô, um índice direito recua enquanto o elemento for maior que o pivô, e os dois elementos encontrados são trocados. Quando os índices se cruzam, o intervalo está dividido em duas partes (separadas por | abaixo), que são ordenadas do mesmo jeito. Intervalos com 1 elemento já estão ordenados.

| Passo | Intervalo | Início, meio, fim → pivô | Trocas | Resultado da partição |
|---|---|---|---|---|
| 1 | [0..5]<br>`22 35 2 1 13 7` | `22 2 7`<br>→ pivô = **7** | 22 ↔ 7<br>35 ↔ 1 | `7 1 2` **\|** `35 13 22` |
| 2 | [0..2]<br>`7 1 2` | `7 1 2`<br>→ pivô = **2** | 7 ↔ 2 | `2 1` **\|** `7` |
| 3 | [0..1]<br>`2 1` | `2 2 1`<br>→ pivô = **2** | 2 ↔ 1 | `1` **\|** `2` |
| 4 | [3..5]<br>`35 13 22` | `35 13 22`<br>→ pivô = **22** | 35 ↔ 22 | `22 13` **\|** `35` |
| 5 | [3..4]<br>`22 13` | `22 22 13`<br>→ pivô = **22** | 22 ↔ 13 | `13` **\|** `22` |

`Resultado final do Quicksort: 1 2 7 13 22 35`

### Heapsort (max-heap)

O vetor é visto como uma árvore binária: o filho esquerdo da posição i está em 2i+1 e o direito em 2i+2. Primeiro o vetor é transformado em um **max-heap** (cada pai é maior que seus filhos), "peneirando" cada pai do último até a raiz. Depois, repetidamente, a raiz (maior valor) é trocada com o último elemento do heap, o heap diminui uma posição e a nova raiz é "peneirada" para baixo.

**Parte 1 — construção do max-heap**

| Passo | Pai peneirado | Trocas | Vetor |
|---|---|---|---|
| — | vetor inicial | — | `22 35 2 1 13 7` |
| 1 | posição 2 (valor 2) | 2 ↔ 7 | `22 35 7 1 13 2` |
| 2 | posição 1 (valor 35) | nenhuma (já é maior que os filhos) | `22 35 7 1 13 2` |
| 3 | posição 0 (valor 22) | 22 ↔ 35 | `35 22 7 1 13 2` |

**Parte 2 — extração do maior elemento** (heap pronto: `35 22 7 1 13 2`)

| Passo | Troca raiz ↔ último do heap | Peneirar a nova raiz | Vetor (ordenado após \|) |
|---|---|---|---|
| 1 | 35 ↔ 2 | 2 ↔ 22<br>2 ↔ 13 | `22 13 7 1 2 \| 35` |
| 2 | 22 ↔ 2 | 2 ↔ 13 | `13 2 7 1 \| 22 35` |
| 3 | 13 ↔ 1 | 1 ↔ 7 | `7 2 1 \| 13 22 35` |
| 4 | 7 ↔ 1 | 1 ↔ 2 | `2 1 \| 7 13 22 35` |
| 5 | 2 ↔ 1 | nenhuma troca | `1 \| 2 7 13 22 35` |

`Resultado final do Heapsort: 1 2 7 13 22 35`

## 2) Mergesort

### Como funciona

O Mergesort usa **divisão e conquista**. (1) **Divide**: parte o vetor ao meio, e continua dividindo cada metade até sobrarem sublistas de 1 elemento, que já estão ordenadas. (2) **Conquista/combina**: volta intercalando (merge) pares de sublistas ordenadas: compara os primeiros elementos das duas sublistas, copia o menor para um vetor auxiliar e avança naquela sublista; quando uma acaba, copia o que sobrou da outra. A árvore de divisões tem cerca de $\log_2 n$ níveis e cada nível gasta O(n) para intercalar, então o tempo é Θ(n log n) em todos os casos (melhor, médio e pior). Precisa de O(n) de memória extra e é estável (elementos iguais mantêm a ordem original).

### Semelhanças e diferenças em relação ao Quicksort

**Semelhanças:** os dois usam divisão e conquista, dividem o problema em duas partes que são ordenadas recursivamente, e têm tempo médio Θ(n log n).

| Característica | Mergesort | Quicksort |
|---|---|---|
| Como divide | Sempre ao meio, pela posição, sem olhar os valores (partes sempre balanceadas) | Pelo pivô, olhando os valores (as partes podem ficar desbalanceadas) |
| Onde está o trabalho | Na volta da recursão: intercalação (merge) das metades ordenadas | Antes da recursão: partição em torno do pivô; depois não há nada a combinar |
| Pior caso | O(n log n) | $O(n^2)$ (pivô ruim, ex.: sempre o menor ou o maior valor) |
| Melhor / caso médio | Ω(n log n) / Θ(n log n) | Ω(n log n) / Θ(n log n) |
| Memória extra | O(n) (vetor auxiliar) | O(log n) (pilha da recursão); ordena no próprio vetor |
| Estabilidade | Estável | Não estável |
| Na prática | Tempo previsível; bom para listas encadeadas e dados em disco | Em vetores costuma ser mais rápido (menos cópias, melhor uso de cache) |

### Execução do Mergesort em {22, 35, 2, 1, 13, 7}

| Etapa | Sublistas |
|---|---|
| Vetor original | `[22 35 2 1 13 7]` |
| Divisão 1 | `[22 35 2] [1 13 7]` |
| Divisão 2 | `[22 35] [2] [1 13] [7]` |
| Divisão 3 | `[22] [35] [2] [1] [13] [7]` |

Depois as sublistas são intercaladas de volta (na ordem em que ocorrem na recursão):

| Intercalação | Comparações (esquerda × direita → copiado) | Resultado |
|---|---|---|
| `[22] + [35]` | 22×35 → 22; sobra: 35 | `22 35` |
| `[22 35] + [2]` | 22×2 → 2; sobra: 22 35 | `2 22 35` |
| `[1] + [13]` | 1×13 → 1; sobra: 13 | `1 13` |
| `[1 13] + [7]` | 1×7 → 1; 13×7 → 7; sobra: 13 | `1 7 13` |
| `[2 22 35] + [1 7 13]` | 2×1 → 1; 2×7 → 2; 22×7 → 7; 22×13 → 13; sobra: 22 35 | `1 2 7 13 22 35` |

`Resultado final do Mergesort: 1 2 7 13 22 35`

### Referências


• T. Cormen, C. Leiserson, R. Rivest, C. Stein. *Introduction to Algorithms* (Merge Sort: seção 2.3; Heapsort: cap. 6; Quicksort: cap. 7).
• R. Sedgewick, K. Wayne. *Algorithms*, 4ª ed. (Mergesort: seção 2.2; Quicksort: seção 2.3).
• D. Knuth. *The Art of Computer Programming*, vol. 3 (Shellsort e sequências de h).